Incident Analysis Report

Linux worker host compromise

By: Mark Rizal Tholib

Summary

The Linux worker host was very likely compromised and used for **XMRig-based cryptomining**, with the threat actor relying on **masquerading, tmpfs staging, and anti-forensic cleanup**. UAC artifacts and memory findings are consistent with an threat actor working from **/dev/shm/kit**, renaming **xmrig** to **top**, modifying **PATH** so that the local binary would be preferred, launching **ssh** with a custom configuration, and then executing the payload as **./top -o 127.0.0.1:3333 -B** before removing the staging directory. UAC artifacts preserved the process residue as **/proc/977/exe -> /dev/shm/kit/top (deleted)** and **/proc/977/cwd -> /dev/shm/kit (deleted)**, while the memory findings confirmed that the payload was indeed executed from that location rather than from a legitimate system binary.

Memory analysis also clarified the attacker's process chain as **sshd -> sh -> python3 -> bash -> ssh**, with the **top** payload later continuing as an orphaned process under **PID 1**. Socket evidence supports an operational model in which **PID 975 (ssh)** opened a local listener on **127.0.0.1:3333** and maintained a connection to **192.168.5.95:22**, while the **top** payload and its related threads communicated with that local endpoint. Taken together, the most plausible model is:

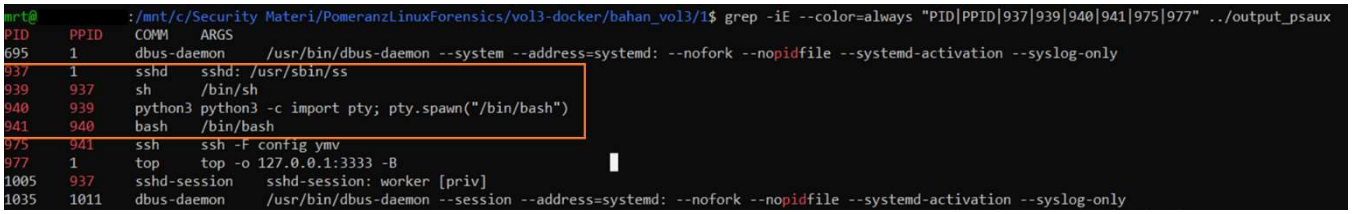
top -> 127.0.0.1:3333 -> ssh helper -> 192.168.5.95:22

The address **192.168.4.35** should still be treated as a **jump host / infrastructure hop**, not as a malicious IP. By contrast, **192.168.5.95** should now be treated as a **suspicious remote SSH endpoint involved in the observed workflow**, although it would still be premature at this stage to label it definitively as the final command-and-control server. The **19:34 onward** session, which resulted in **sudo /bin/bash** followed by execution of **./uac**, should continue to be treated as **acquisition-related contamination**, not as part of the primary attacker activity.

ATT&CK-Aligned Attack Narrative

1. Initial Access

The initial access path to the host was **SSH**, and known access to the system transited the jump host **192.168.4.35**. As such, that address should be treated as **infrastructure**, not as a malicious IOC. Memory findings now provide a clearer and more defensible reconstruction of the attacker's interactive session than was possible from the earlier UAC-only review. Specifically, the shell activity can be traced back to the SSH service process: **sshd (937)** spawned **sh (939)**, which in turn spawned **python3 (940)** and **bash (941)**. This establishes that the attacker's interactive workflow was rooted in an SSH session and confirms a much stronger process chain than was previously available from host triage alone.



2. Execution

Memory analysis now provides a much clearer picture of execution than was available in the earlier draft. The first host-side clues appear in the UAC process-path artifacts, which show a suspicious process cluster associated with the threat actor's activity:

- **PID 937** → /usr/sbin/sshd
- **PID 939** → /usr/bin/dash
- **PID 940** → /usr/bin/python3.13
- **PID 941** → /usr/bin/bash
- **PID 975** → /usr/bin/ssh
- **PID 977** → /dev/shm/kit/top (deleted)

This is significant because the suspicious payload was not executing from a normal system location such as /usr/bin/top, but instead from **/dev/shm/kit/top**, with the executable already appearing as **deleted** at the time of collection. That alone strongly suggests execution from a volatile staging location rather than from a legitimate system-installed program.

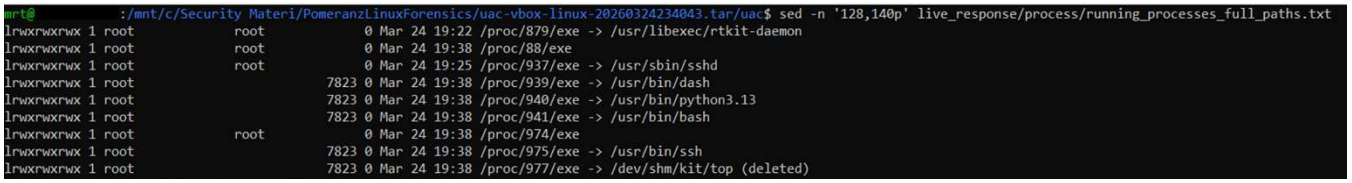


Figure E-1. UAC process-path artifacts showing the suspicious process cluster, including ssh and the top payload executing from /dev/shm/kit/top (deleted).

A closer view of the deleted-path artifacts strengthens that interpretation. UAC shows that:

- **/proc/975/cwd -> /dev/shm/kit (deleted)**
- **/proc/977/cwd -> /dev/shm/kit (deleted)**
- **/proc/977/exe -> /dev/shm/kit/top (deleted)**

These entries indicate that both the SSH helper process and the payload were still running even though the staging directory and executable path had already been removed. This is consistent with a deliberate anti-forensic cleanup step performed after launch.

Figure E-2. Zoomed view of the deleted working-directory and executable-path artifacts associated with the SSH helper and the top payload.

The working-directory evidence provides additional context for how the execution unfolded. UAC shows that PID 941 (bash) had cwd -> /dev/shm, while both PID 975 (ssh) and PID 977 (top) were tied to /dev/shm/kit (deleted). This supports a workflow in which the threat actor moved into a tmpfs staging area, launched the helper and payload from there, and then removed the staging directory after execution.

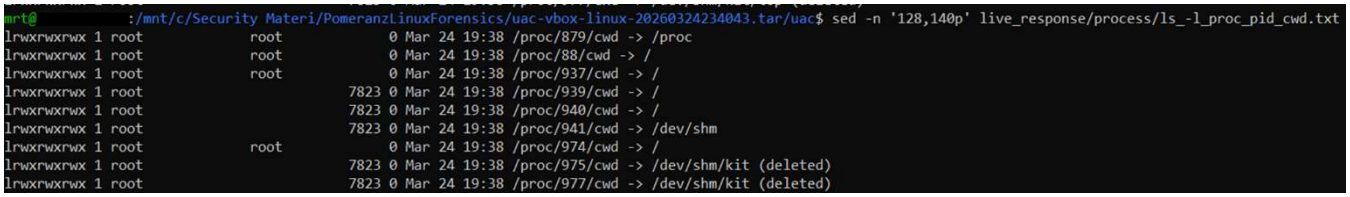


Figure E-3. UAC working-directory artifacts showing the attacker workflow moving through /dev/shm and /dev/shm/kit (deleted).

Memory analysis then confirms the attacker-controlled process chain and captures the exact command-line behavior used to promote the shell and launch the SSH helper and disguised payload. The attacker-controlled process chain can be reconstructed as:

sshd (937) -> sh (939) -> python3.13 (940) -> bash (941) -> ssh (975)

In parallel, the suspicious payload **top (PID 977)** appears as an orphaned process under **PID 1**, indicating that it continued running after the launching shell context detached or exited. While Linux can legitimately orphan background tasks, that behavior is significant here because the process is tied to a deleted tmpfs staging path, a disguised process name, and the CPU-burn symptom reported by the SOC.

Execution is now directly confirmed in memory. **PsAux** captures the exact command line `**python3 -c import pty; pty.spawn("/bin/bash")**` for **PID 940**, showing that the shell was explicitly promoted into an interactive Bash session. The **in-memory Bash history** for **PID 941** then records the threat actor's workflow in /dev/shm/kit:

- cd /dev/shm/kit
- ls
- mv xmrig top
- export PATH=.:$PATH
- ssh -F config ymv
- bg
- top -o 127.0.0.1:3333 -B
- cd ..
- rm -rf kit
- ps -ef | grep top

This sequence reflects a deliberate workflow: stage the toolkit in tmpfs, rename the miner for masquerading, modify PATH so the current directory is prioritized, launch the SSH helper, execute the disguised payload in background mode, and then remove the staging directory. As noted in Hal's course material, the Bash plugin is especially valuable because in-memory command history can preserve commands from the active session that may never have been written to disk, while also retaining per-command timestamps.

```
$ sudo vol -f mem/avml.lime linux.bash.Bash
Volatility 3 Framework 2.26.0
Progress:  100.00               Stacking attempts finished
PID     Process CommandTime     Command

941     bash    2026-03-24 23:26:26.000000 UTC  rest
941     bash    2026-03-24 23:26:30.000000 UTC  reset
941     bash    2026-03-24 23:26:44.000000 UTC  echo $SHELL
941     bash    2026-03-24 23:26:54.000000 UTC  id
941     bash    2026-03-24 23:27:25.000000 UTC  cd /dev/shm/kit
941     bash    2026-03-24 23:27:26.000000 UTC  ls
941     bash    2026-03-24 23:27:51.000000 UTC  mv xmrig top
941     bash    2026-03-24 23:27:59.000000 UTC  export PATH=.:$PATH
941     bash    2026-03-24 23:28:17.000000 UTC  ssh -F config ymv
941     bash    2026-03-24 23:28:28.000000 UTC  bg
941     bash    2026-03-24 23:29:09.000000 UTC  top -o 127.0.0.1:3333 -B
941     bash    2026-03-24 23:29:15.000000 UTC  cd ..
941     bash    2026-03-24 23:29:19.000000 UTC  rm -rf kit
941     bash    2026-03-24 23:29:24.000000 UTC  ps -ef | grep top
941     bash    2026-03-24 23:29:36.000000 UTC  ls
```

Additional UAC and memory artifacts establish the runtime context of the payload:

- **PID 977** maps to **/dev/shm/kit/top**
- the process environment shows **PWD=/dev/shm/kit**
- the process environment shows **_ = ./top**
- the process environment shows **PATH=.:...**

Taken together, these artifacts confirm that the payload was executed as **./top** from **/dev/shm/kit**, rather than as the legitimate system utility **/usr/bin/top**. This materially strengthens the masquerading and anti-forensic narrative.

```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|937|939|940|941|975|977" output_pstree
OFFSET (V)      PID     TID     PPID    COMM
* 0x8c7cc67e1980        937     937     1       sshd
** 0x8c7cc8278000       939     939     937     sh
*** 0x8c7cc67e4c80      940     940     939     python3
**** 0x8c7cc6011980     941     941     940     bash
***** 0x8c7cc6014c80    975     975     941     ssh
** 0x8c7cc81b1980       1005    1005    937     sshd-session
* 0x8c7cc8356600        977     977     1       top
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|937|939|940|941|975|977" output_pslist
OFFSET (V)      PID     TID     PPID    COMM    UID     GID     EUID    EGID    CREATION TIME   File output
0x8c7cc0d79980  310     310     1       systemd-journal 0       0       0       0       2026-03-24 23:23:02.093775 UTC  Disabled
0x8c7cc67e1980  937     937     1       sshd    0       0       0       0       2026-03-24 23:25:37.328087 UTC  Disabled
0x8c7cc8278000  939     939     937     sh      0       7823    0       7823    2026-03-24 23:26:07.280164 UTC  Disabled
0x8c7cc67e4c80  940     940     939     python3 0       7823    0       7823    2026-03-24 23:26:22.541222 UTC  Disabled
0x8c7cc6011980  941     941     940     bash    0       7823    0       7823    2026-03-24 23:26:22.581124 UTC  Disabled
0x8c7cc6014c80  975     975     941     ssh     0       7823    0       7823    2026-03-24 23:28:32.499082 UTC  Disabled
0x8c7cc8356600  977     977     1       top     0       7823    0       7823    2026-03-24 23:29:24.368597 UTC  Disabled
0x8c7cc81b1980  1005    1005    937     sshd-session    0       0       0       0       2026-03-24 23:34:37.655750 UTC  Disabled
0x8c7cc3e4e600  1035    1035    1011    dbus-daemon     1000    1000    1000    1000    2026-03-24 23:34:42.371939 UTC  Disabled
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|937|939|940|941|975|977" ../output_psaux
PID     PPID    COMM    ARGS
695     1       dbus-daemon     /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
937     1       sshd    sshd: /usr/sbin/ss
939     937     sh      /bin/sh
940     939     python3 python3 -c import pty; pty.spawn("/bin/bash")
941     940     bash    /bin/bash
975     941     ssh     ssh -F config ymv
977     1       top     top -o 127.0.0.1:3333 -B
1005    937     sshd-session    sshd-session: worker [priv]
1035    1011    dbus-daemon     /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
```

Figure E-4. Memory-resident process hierarchy and command-line evidence showing the attacker shell chain, the promoted Bash session, the SSH helper, and the orphaned top payload under PID 1.

Overall, the execution evidence is now substantially stronger than in the earlier UAC-only draft. UAC established the deleted tmpfs execution path, while memory analysis confirmed the promoted shell, the

attacker-controlled process chain, the exact threat actor command sequence, and the runtime context of the disguised payload.

3. Persistence

Persistence has **not** been established from the artifacts reviewed to date. Initial review of **systemd units, cron entries, modules-load, insmod, and modprobe-related artifacts** did not produce convincing evidence of a persistence mechanism. Likewise, the memory analysis completed so far does not show a confirmed loaded implant that can be tied to persistence.

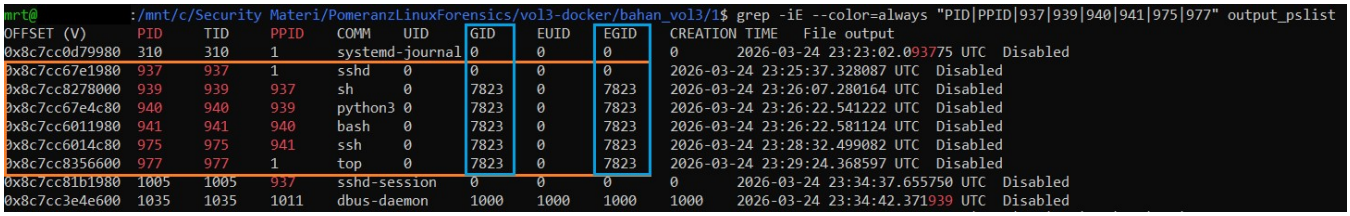
At this stage, the compromise is best characterized as an **active userland operation** rather than a fully reconstructed persistent foothold. In other words, the available evidence is sufficient to explain the observed execution, tunneling, and anti-forensic behavior, but it does not yet support a firm conclusion that the attacker established persistence on the host.

4. Privilege Escalation / Root Context

Root context is now confirmed for the attacker-side process chain. **PsList** and **PsAux** show that **PIDs 939, 940, 941, 975, and 977** were all running as **UID 0**, indicating that the observed malicious workflow was operating with root privileges. Those same artifacts also show an unusual group context, with **GID 7823** rather than the more typical root group **GID 0**. This is noteworthy and should be documented, but it does not by itself explain how root access was obtained.

At this stage, the available evidence is sufficient to confirm that the attacker's activity was executed with **root-level privileges**, but it is still **not sufficient to reconstruct the exact mechanism** used to obtain that level of access before the observed command sequence began. In other words, **root context is confirmed, but the privilege-escalation path itself remains unresolved**.

The later **worker -> sudo -> /bin/bash -> uac** chain observed at **19:34** belongs to the acquisition session and must remain excluded from attacker analysis. That sequence reflects post-incident collection activity, not the original attacker escalation path.



5. Defense Evasion

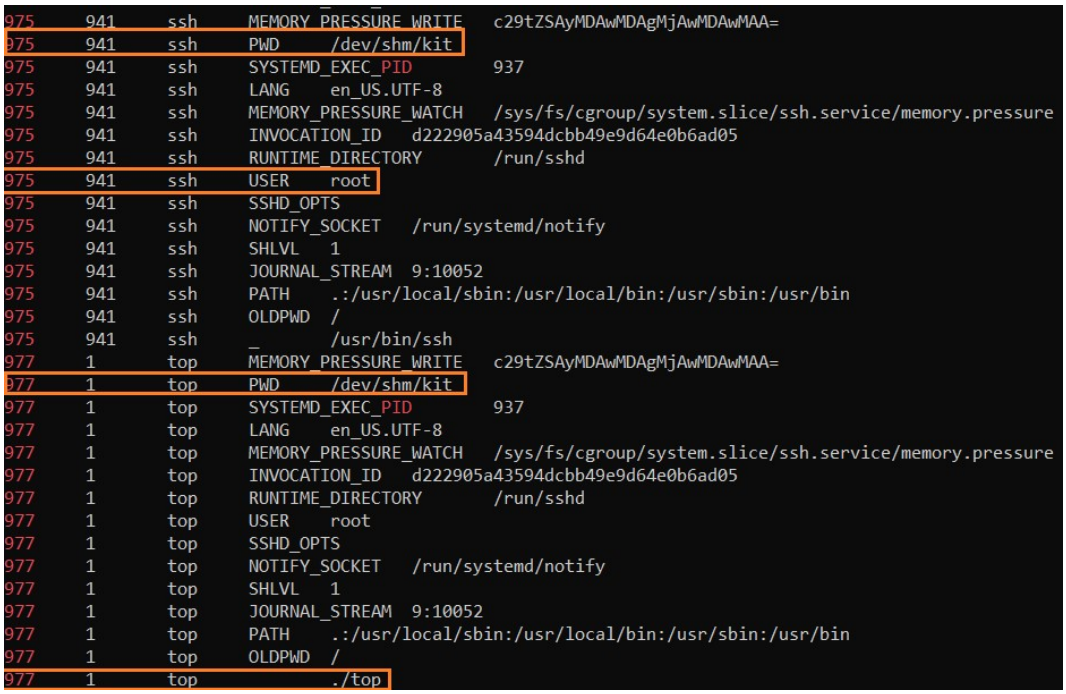
Defense evasion is one of the strongest confirmed themes in this case.

The threat actor used several layered techniques to reduce visibility, minimize filesystem residue, and complicate live-response triage. First, the toolkit was staged in **/dev/shm/kit**, a tmpfs-backed location that exists in memory and is lost on reboot. Using this path allowed the threat actor to run tooling from a volatile staging area rather than from a normal system location.

Second, the payload was **masqueraded** by renaming **xmrig** to **top**, causing the running process to appear more benign during casual review. This was reinforced by **PATH manipulation**, specifically **export PATH=.:$PATH**,

which ensured that invoking top would prefer the local binary in the current directory over the legitimate system utility in /usr/bin.

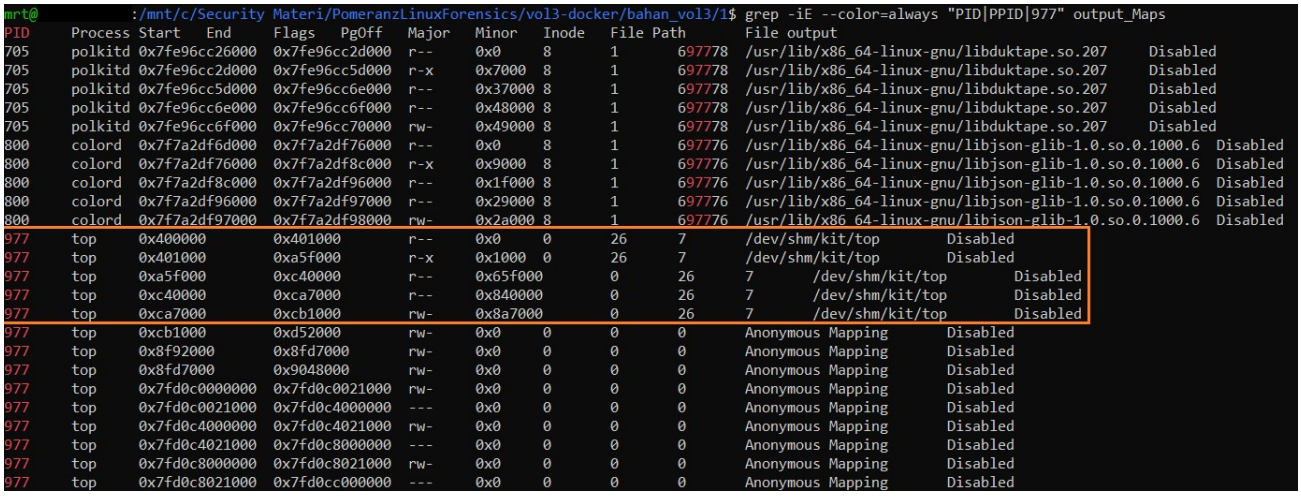
Further memory artifacts confirm that runtime context. Process environment data for the relevant processes shows **PWD=/dev/shm/kit**, **PATH=.:...**, and for the payload, **_ = ./top**, directly supporting the conclusion that the suspicious binary was executed as **./top** from the staging directory rather than as the legitimate system binary.



```
975     941     ssh     MEMORY_PRESSURE_WRITE   c29tZSAyMDAwMDAgMjAwMDAwMAA=
975     941     ssh     PWD     /dev/shm/kit
975     941     ssh     SYSTEMD_EXEC_PID        937
975     941     ssh     LANG    en_US.UTF-8
975     941     ssh     MEMORY_PRESSURE_WATCH   /sys/fs/cgroup/system.slice/ssh.service/memory.pressure
975     941     ssh     INVOCATION_ID   d222905a43594dcbb49e9d64e0b6ad05
975     941     ssh     RUNTIME_DIRECTORY       /run/sshd
975     941     ssh     USER    root
975     941     ssh     SSHD_OPTS
975     941     ssh     NOTIFY_SOCKET   /run/systemd/notify
975     941     ssh     SHLVL   1
975     941     ssh     JOURNAL_STREAM  9:10052
975     941     ssh     PATH    .:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
975     941     ssh     OLDPWD  /
975     941     ssh     _       /usr/bin/ssh
977     1       top     MEMORY_PRESSURE_WRITE   c29tZSAyMDAwMDAgMjAwMDAwMAA=
977     1       top     PWD     /dev/shm/kit
977     1       top     SYSTEMD_EXEC_PID        937
977     1       top     LANG    en_US.UTF-8
977     1       top     MEMORY_PRESSURE_WATCH   /sys/fs/cgroup/system.slice/ssh.service/memory.pressure
977     1       top     INVOCATION_ID   d222905a43594dcbb49e9d64e0b6ad05
977     1       top     RUNTIME_DIRECTORY       /run/sshd
977     1       top     USER    root
977     1       top     SSHD_OPTS
977     1       top     NOTIFY_SOCKET   /run/systemd/notify
977     1       top     SHLVL   1
977     1       top     JOURNAL_STREAM  9:10052
977     1       top     PATH    .:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
977     1       top     OLDPWD  /
977     1       top     ./top
```

Figure DE-1. Process environment evidence showing that both the SSH helper and the payload executed from /dev/shm/kit, with current-directory prioritization in PATH and the payload invoked as ./top.

This interpretation is further strengthened by memory mapping evidence. **PID 977** maps to **/dev/shm/kit/top**, confirming that the running payload was backed by the tmpfs staging path. This is a key point in the evasion narrative because it ties the disguised process directly to the volatile attacker-controlled directory.



```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|977" output_Maps
PID     Process Start   End     Flags   PgOff   Major   Minor   Inode   File Path       File output
705     polkitd 0x7fe96cc26000  0x7fe96cc2d000  r--     0x0     8       1       697778  /usr/lib/x86_64-linux-gnu/libduktape.so.207     Disabled
705     polkitd 0x7fe96cc2d000  0x7fe96cc5d000  r-x     0x7000  8       1       697778  /usr/lib/x86_64-linux-gnu/libduktape.so.207     Disabled
705     polkitd 0x7fe96cc5d000  0x7fe96cc6e000  r--     0x37000 8       1       697778  /usr/lib/x86_64-linux-gnu/libduktape.so.207     Disabled
705     polkitd 0x7fe96cc6e000  0x7fe96cc6f000  r--     0x48000 8       1       697778  /usr/lib/x86_64-linux-gnu/libduktape.so.207     Disabled
705     polkitd 0x7fe96cc6f000  0x7fe96cc70000  rw-     0x49000 8       1       697778  /usr/lib/x86_64-linux-gnu/libduktape.so.207     Disabled
800     colord  0x7f7a2df6d000  0x7f7a2df76000  r--     0x0     8       1       697776  /usr/lib/x86_64-linux-gnu/libjson-glib-1.0.so.0.1000.6  Disabled
800     colord  0x7f7a2df76000  0x7f7a2df8c000  r-x     0x9000  8       1       697776  /usr/lib/x86_64-linux-gnu/libjson-glib-1.0.so.0.1000.6  Disabled
800     colord  0x7f7a2df8c000  0x7f7a2df96000  r--     0x1f000 8       1       697776  /usr/lib/x86_64-linux-gnu/libjson-glib-1.0.so.0.1000.6  Disabled
800     colord  0x7f7a2df96000  0x7f7a2df97000  r--     0x29000 8       1       697776  /usr/lib/x86_64-linux-gnu/libjson-glib-1.0.so.0.1000.6  Disabled
800     colord  0x7f7a2df97000  0x7f7a2df98000  rw-     0x2a000 8       1       697776  /usr/lib/x86_64-linux-gnu/libjson-glib-1.0.so.0.1000.6  Disabled
977     top     0x400000        0x401000        r--     0x0     0       26      7       /dev/shm/kit/top        Disabled
977     top     0x401000        0xa5f000        r-x     0x1000  0       26      7       /dev/shm/kit/top        Disabled
977     top     0xa5f000        0xc40000        r--     0x65f000        0       26      7       /dev/shm/kit/top        Disabled
977     top     0xc40000        0xca7000        r--     0x840000        0       26      7       /dev/shm/kit/top        Disabled
977     top     0xca7000        0xcb1000        rw-     0x8a7000        0       26      7       /dev/shm/kit/top        Disabled
977     top     0xcb1000        0xd52000        rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x8f92000       0x8fd7000       rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x8fd7000       0x9048000       rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c0000000  0x7fd0c0021000  rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c0021000  0x7fd0c4000000  ---     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c4000000  0x7fd0c4021000  rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c4021000  0x7fd0c8000000  ---     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c8000000  0x7fd0c8021000  rw-     0x0     0       0       0       Anonymous Mapping       Disabled
977     top     0x7fd0c8021000  0x7fd0cc000000  ---     0x0     0       0       0       Anonymous Mapping       Disabled
```

Figure DE-2. Memory mapping evidence showing that PID 977 was backed by /dev/shm/kit/top, confirming execution from the tmpfs staging path.

The threat actor also used **background execution and detachment** to reduce obvious linkage between the interactive shell and the running payload. The payload was launched as **top -o 127.0.0.1:3333 -B**, and later appeared as an orphaned process under **PID 1**. While Linux can legitimately orphan background tasks, in this context that behavior is significant because the process was tied to a deleted tmpfs path, a disguised name, and the suspicious runtime chain reconstructed from memory.

In addition, the staging directory was deliberately removed after launch. The command **rm -rf kit** explains why later UAC artifacts showed:

- **cwd -> /dev/shm/kit (deleted)**
- **exe -> /dev/shm/kit/top (deleted)**

Taken together, these behaviors form a coherent evasion workflow: run from tmpfs, disguise the payload, manipulate PATH, detach execution, and clean up the staging directory after launch.

Figure DE-3. Process-reference artifact showing PID 977 under proc/1/children, consistent with detached or orphaned execution after launch.

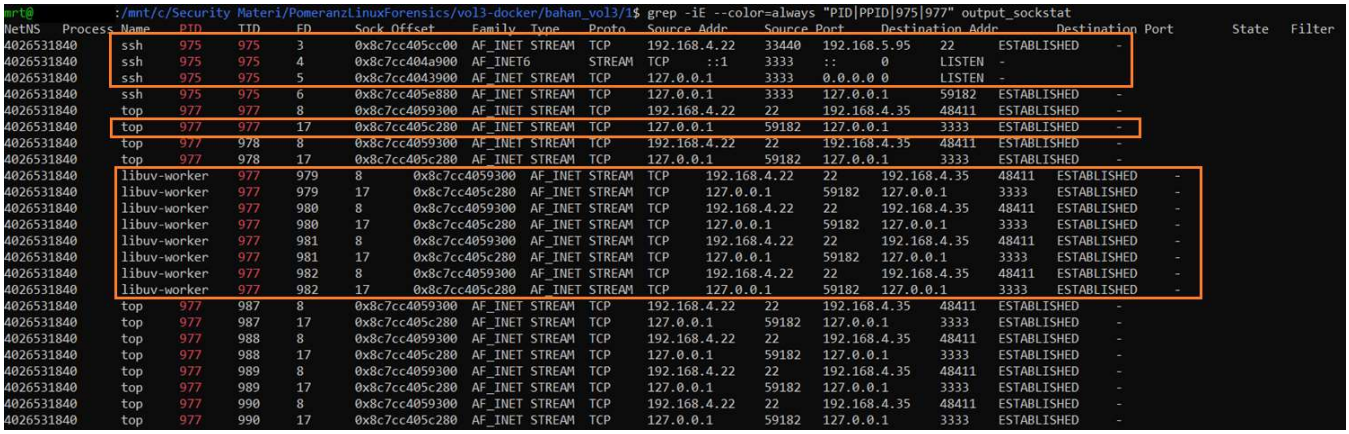
At this stage, the evidence supports **userland anti-forensic execution and visibility reduction**, not a confirmed **kernel-level rootkit**. While artifacts such as **rk.so** remain suspicious and should be documented, the currently reviewed memory plugins do not yet establish that a kernel-mode persistence or hiding implant was actively loaded.

6. Command and Control / Tunneling

Memory-based socket evidence now provides a much clearer picture of the threat actor's communication path than was available in the earlier draft.

The key process is **PID 975 (ssh)**, which runs from working directory **/dev/shm/kit**, maintains an outbound connection to **192.168.5.95:22**, and opens local listeners on **127.0.0.1:3333** and **::1:3333**. At the same time, **PID 977 (top)** and its related worker threads connect to the local endpoint **127.0.0.1:3333**. This strongly supports a tunneled execution model in which the disguised mining payload does not connect outward directly, but instead communicates through a locally exposed SSH-assisted channel.

This interpretation is further supported by the presence of multiple **libuv-worker** threads associated with **PID 977**, which also show communication with the same local endpoint. This indicates that use of **127.0.0.1:3333** was not limited to the parent top process alone, but formed part of the broader runtime behavior of the payload and its worker threads.



```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|975|977" output_sockstat
NetNS   Process Name    PID     TID     FD      Sock Offset     Family  Type    Proto   Source Addr     Source Port     Destination Addr        Destination Port        State   Filter
4026531840      ssh     975     975     3       0x8c7cc405cc00  AF_INET STREAM  TCP     192.168.4.22    33440   192.168.5.95    22      ESTABLISHED     -
4026531840      ssh     975     975     4       0x8c7cc404a900  AF_INET6        STREAM  TCP     ::1     3333    ::      0       LISTEN  -
4026531840      ssh     975     975     5       0x8c7cc4043900  AF_INET STREAM  TCP     127.0.0.1       3333    0.0.0.0 0       LISTEN  -
4026531840      ssh     975     975     6       0x8c7cc405e880  AF_INET STREAM  TCP     127.0.0.1       3333    127.0.0.1       59182   ESTABLISHED     -
4026531840      top     977     977     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     977     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     978     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     978     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     979     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     979     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     980     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     980     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     981     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     981     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     982     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     982     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     987     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     987     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     988     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     988     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     989     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     989     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     990     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     990     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
```

Figure C2-1. Memory-resident socket evidence showing the SSH helper (PID 975) listening on 127.0.0.1:3333, maintaining an outbound connection to 192.168.5.95:22, and the top payload (PID 977) together with related libuv-worker threads communicating with the local tunnel endpoint.

The most defensible reconstruction at this stage is:

top (PID 977) -> 127.0.0.1:3333 -> ssh helper (PID 975) -> 192.168.5.95:22

This model explains why **port 22** remains operationally central to the case, why the payload appears to target **localhost** rather than a direct external destination, and why **192.168.5.95** should be treated as a **suspicious remote SSH endpoint involved in the observed workflow**.

At the same time, **192.168.4.35** remains a **jump host / infrastructure hop**, not a malicious IP in its own right. That distinction is important, because the evidence now supports two different SSH roles in the case:

1. **192.168.4.35** as the administrative access path into the host, and
2. **192.168.5.95** as the suspicious remote SSH endpoint associated with the helper/tunneling workflow.

The current evidence is therefore strong enough to support an **SSH-assisted tunneling model**, even though the report should still avoid overstating **192.168.5.95** as a definitively proven final command-and-control server.

7. Impact

The observed impact is consistent with **resource hijacking for cryptomining**. The threat actor explicitly renamed **xmrig** to **top**, and both **PsAux** and in-memory Bash history capture the payload being launched as **top -o 127.0.0.1:3333 -B**. The process later appeared as **PID 977** under **PID 1** and remained consistently associated with the same suspicious runtime context, including execution from /dev/shm/kit, detached execution, and communication through the SSH-assisted local tunnel.

The host also showed repeated **crash/reboot events** that overlapped the main suspicious activity window, further supporting the conclusion that the system was materially affected by the malicious workflow. Taken together, the available evidence supports the assessment that the host was not merely accessed, but was actively used in a way that consumed system resources and contributed to operational instability.

Timeline

Note: The timeline below intentionally preserves mixed time bases where necessary. Session and package-install rows remain uac-derived because timezone normalization across those artifacts has not been fully reconciled. Memory-derived process and command events are presented in UTC exactly as reported by Volatility 3.

Timestamp	Source Evidence	User / TTY	Source IP	Event	Evidence / Assessment
2026-03-24 11:52:59	uac-derived	N/A / N/A	N/A	Account `worker` created	Installer syslog records adduser, groupadd, useradd, home directory creation, and password change for worker.
2026-03-24 11:55:25 - 12:56:39	uac-derived	worker / tty2	local	First local login session for `worker`	`last -a -F` / `last wtmp.db` show `worker` logged in locally almost immediately after provisioning.
2026-03-24 11:58:39 - 12:06:23	uac-derived	worker / pts/1	192.168.4.35	Remote SSH session via jump host	`last -a -F` / `last wtmp.db` show `worker` early remote access via the jump host.
2026-03-24 12:02:12 - 12:56:39	uac-derived	worker / pts/2	192.168.4.35	Second remote SSH session via jump host	`last -a -F` / `last wtmp.db` show `worker` activity continued during early setup period.
2026-03-24 12:02:56	uac-derived	worker / N/A	N/A	apt install build-essential	apt history.log attributes the request to worker (UID 1000).
2026-03-24 12:09:02	uac-derived	worker / N/A	N/A	apt install git	apt history.log attributes the request to worker.
2026-03-24 12:48:28	uac-derived	worker / N/A	N/A	apt install libpam0g-dev	Sensitive development package installed.
2026-03-24 12:49:46	uac-derived	worker / N/A	N/A	apt install libgcrypt-dev	Crypto-related development package installed.
2026-03-24 12:51:42	uac-derived	worker / N/A	N/A	apt install nasm	Low-level tooling installed.
2026-03-24 12:56:39	uac-derived	N/A / system boot	N/A	Crash / reboot #1	`last -a -F` / `last wtmp.db` record reboot with - crash.
2026-03-24 13:07:12 - 13:14:13	uac-derived	worker / pts/0	192.168.4.35	Short remote SSH session via jump host	`last -a -F` / `last wtmp.db` show `worker` activity resumed after first crash.
2026-03-24 14:54:59 - 18:15:05	uac-derived	worker / pts/1	192.168.4.35	Main remote SSH session via jump host	`last -a -F` / `last wtmp.db` show longest pre-acquisition worker session; strongest suspicious window in uac-derived logs.

Timestamp	Source Evidence	User / TTY	Source IP	Event	Evidence / Assessment
2026-03-24 18:03:06 - 18:15:05	uac-derived	worker / pts/3	192.168.4.35	Additional remote SSH session	`last -a -F` / `last wtmp.db` show concurrent session shortly before second crash.
2026-03-24 18:15:00	uac-derived	N/A / system boot	N/A	Crash / reboot #2	`last -a -F` / `last wtmp.db` show directly overlaps the main suspicious uac-derived session window.
2026-03-24 19:16:24	uac-derived	N/A / system boot	N/A	Crash / reboot #3	`last -a -F` / `last wtmp.db` show additional reboot with - crash.
2026-03-24 19:25:30	uac-derived	root / N/A	N/A	sshd listener active	`ps` / `lsof` show `sshd` running and listening on port 22 after reboot.
2026-03-24 19:34:32 - ongoing	uac-derived	worker / pts/0	192.168.4.35	Acquisition-related worker session	`last -a -F`, `who -T`, and `loginctl_user-status_worker` show `worker` active on `pts/0` from the jump host. This session is treated as acquisition-related (UAC) contamination.
2026-03-24 19:34:57	uac-derived	worker / pts/0	N/A	sudo /bin/bash	`loginctl_user-status_worker.txt` records `TTY=pts/0 ; USER=root ; COMMAND=/bin/bash`. This is part of the acquisition chain, not attacker evidence
2026-03-24 19:34:59 onward	uac-derived	root / pts/1	N/A	Root shell in acquisition session	`ps -efl` shows PID 1079 `/bin/bash` on `pts/1` as part of the `sudo` chain.
2026-03-24 19:38	uac-derived	root / pts/1	N/A	./uac execution	`ps -efl` shows `/bin/sh ./uac -a memory_dump/avml.yaml -p ir_triage /root`. This event is excluded from the attacker timeline.
2026-03-24 23:26:07 UTC	memory-derived	root / pts/2	192.168.4.35	sshd spawned sh (PID 939)	Pstree and PsList confirm attacker-side shell chain begins here.
2026-03-24 23:26:22 UTC	memory-derived	root / pts/2	192.168.4.35	python3 executed pty.spawn("/bin/bash")	PsAux captures exact command line: python3 -c import pty; pty.spawn("/bin/bash").
2026-03-24 23:27:25 UTC	memory-derived	root / pts/2	N/A	cd /dev/shm/kit	Bash history confirms threat actor moved into tmpfs staging directory.
2026-03-24 23:27:51 UTC	memory-derived	root / pts/2	N/A	mv xmrig top	Bash history confirms miner masquerading step.

Timestamp	Source Evidence	User / TTY	Source IP	Event	Evidence / Assessment
2026-03-24 23:27:59 UTC	memory-derived	root / pts/2	N/A	export PATH=.:$PATH	Bash history and Envars support current-directory precedence.
2026-03-24 23:28:17 UTC	memory-derived	root / pts/2	N/A	ssh -F config ymv	Bash history shows custom-config SSH helper launch.
2026-03-24 23:28:32 UTC	memory-derived	root / pts/2	192.168.5.95	ssh helper process started (PID 975)	PsAux, Envars, and Sockstat confirm /usr/bin/ssh started from /dev/shm/kit and connected to 192.168.5.95:22.
2026-03-24 23:29:09 UTC	memory-derived	root / pts/2	127.0.0.1:3333	top -o 127.0.0.1:3333 -B	Bash history confirms the disguised payload launch.
2026-03-24 23:29:19 UTC	memory-derived	root / pts/2	N/A	rm -rf kit	Bash history confirms cleanup / anti-forensic deletion of staging directory.
2026-03-24 23:29:24 UTC	memory-derived	root / orphaned	N/A	top payload became PID 977 under PPID 1	PsAux / PsList show top as orphaned under PID 1 with deleted tmpfs backing path.

In practical terms, the most relevant attacker activity is bounded by the pre-acquisition worker sessions and the later memory-confirmed shell and payload activity. The strongest window for primary malicious activity remains the pre-acquisition period ending at the 18:15 crash/reboot, while the 19:34 onward chain is treated as acquisition-related contamination and excluded from attacker attribution. This distinction is important because the UAC-derived session data and the memory-confirmed attacker workflow describe two different phases of activity on the host.

Findings

Finding 1 — Disguised XMRig payload executed as top

Memory evidence shows that the threat actor explicitly renamed xmrig to top and then launched it as top -o 127.0.0.1:3333 -B. The resulting process ran as **PID 977**, under **PPID 1**, with **no interactive TTY** and **99% CPU utilization**. Taken together, these artifacts provide the strongest single line of evidence that the host was being used for **cryptomining**, with **XMRig deliberately disguised as top**.

```
$ sudo vol -f mem/avml.lime linux.bash.Bash
Volatility 3 Framework 2.26.0
Progress:  100.00               Stacking attempts finished
PID     Process CommandTime     Command

941     bash    2026-03-24 23:26:26.000000 UTC  rest
941     bash    2026-03-24 23:26:30.000000 UTC  reset
941     bash    2026-03-24 23:26:44.000000 UTC  echo $SHELL
941     bash    2026-03-24 23:26:54.000000 UTC  id
941     bash    2026-03-24 23:27:25.000000 UTC  cd /dev/shm/kit
941     bash    2026-03-24 23:27:26.000000 UTC  ls
941     bash    2026-03-24 23:27:51.000000 UTC  mv xmrig top
941     bash    2026-03-24 23:27:59.000000 UTC  export PATH=.:$PATH
941     bash    2026-03-24 23:28:17.000000 UTC  ssh -F config ymv
941     bash    2026-03-24 23:28:28.000000 UTC  bg
941     bash    2026-03-24 23:29:09.000000 UTC  top -o 127.0.0.1:3333 -B
941     bash    2026-03-24 23:29:15.000000 UTC  cd ..
941     bash    2026-03-24 23:29:19.000000 UTC  rm -rf kit
941     bash    2026-03-24 23:29:24.000000 UTC  ps -ef | grep top
941     bash    2026-03-24 23:29:36.000000 UTC  ls
```

Figure F1-1. In-memory Bash history showing the threat actor renaming xmrig to top and launching the payload with top -o 127.0.0.1:3333 -B.

```
49724851 3858883590 root@vbox:/dev/shm# ps -ef | grep top
49724852 3858883629 root         977       1 99 19:29 ?        00:00:53 top -o 127.0.0.1:3333 -B
49724853 3858883707 root         993     941  0 19:29 pts/2    00:00:00 grep top
```

Figure F1-2. Process evidence showing the disguised top payload running as PID 977 under PID 1, with no interactive TTY and high CPU utilization.

Finding 2 — Interactive shell promotion confirmed in memory

Memory analysis directly confirms that the attacker promoted the initial shell into a more usable interactive Bash session. **PsAux** captures the exact command line for **PID 940** as `python3 -c import pty; pty.spawn("/bin/bash")`, while the surrounding process chain shows this activity rooted in the SSH service process. This provides direct host-side confirmation of the shell-promotion activity that was previously inferred from the network alert.

```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|937|939|940|941|975|977" ../output_psaux
PID     PPID    COMM    ARGS
695     1       dbus-daemon     /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
937     1       sshd    sshd: /usr/sbin/ss
939     937     sh      /bin/sh
940     939     python3 python3 -c import pty; pty.spawn("/bin/bash")
941     940     bash    /bin/bash
975     941     ssh     ssh -F config ymv
977     1       top     top -o 127.0.0.1:3333 -B
1005    937     sshd-session    sshd-session: worker [priv]
1035    1011    dbus-daemon     /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
```

Figure F2-1. Memory-resident process and command-line evidence showing `python3 -c import pty; pty.spawn("/bin/bash")` for PID 940, confirming promotion of the SSH-derived shell into an interactive Bash session.

Finding 3 — Suspicious payload executed from /dev/shm/kit

Both **UAC** and **memory analysis** corroborate the same execution path. UAC artifacts show deleted working-directory and executable paths tied to /dev/shm/kit, while **Maps** and **Envars** confirm that **PID 977** was backed by /dev/shm/kit/top and executed as ./top from that staging directory. Taken together, these artifacts establish that the payload was launched from a **tmpfs-backed staging location**, not from a legitimate system binary path.

Figure F3-1. UAC process artifacts showing the deleted working-directory and executable paths associated with the suspicious payload and SSH helper.

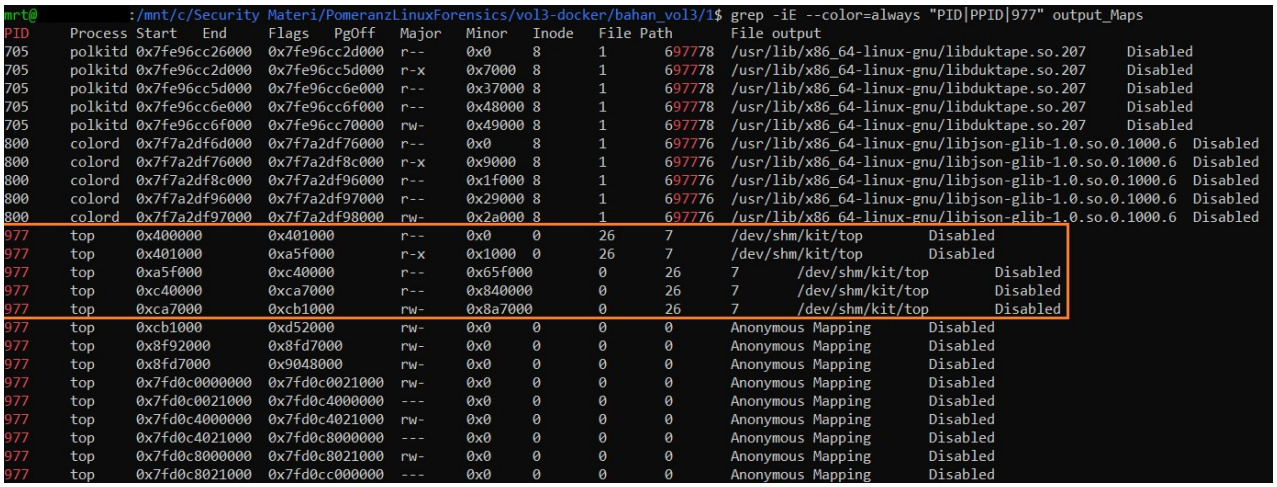


Figure F3-2. Memory mapping evidence showing that PID 977 was backed by /dev/shm/kit/top, confirming execution from the tmpfs staging path.

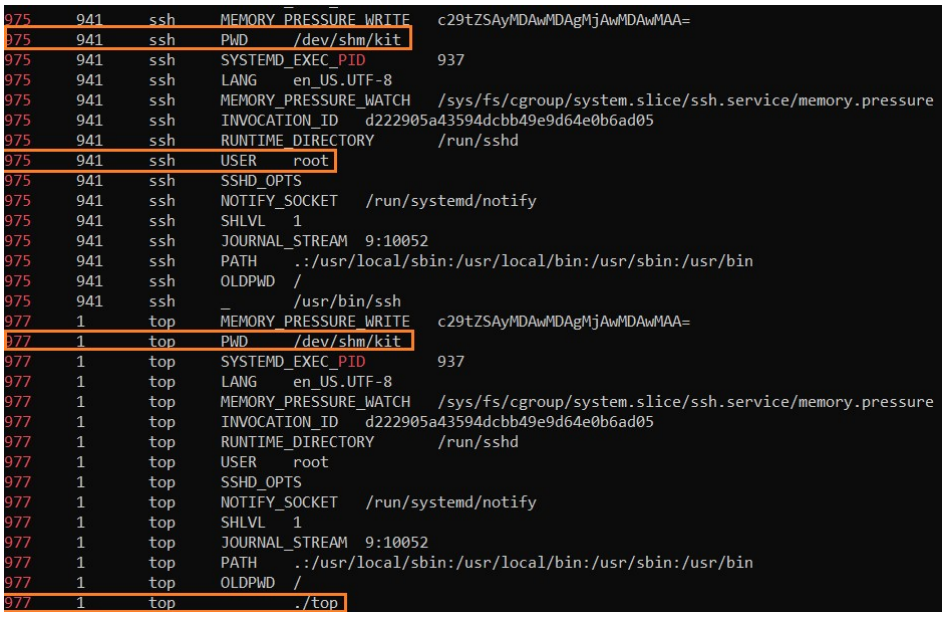
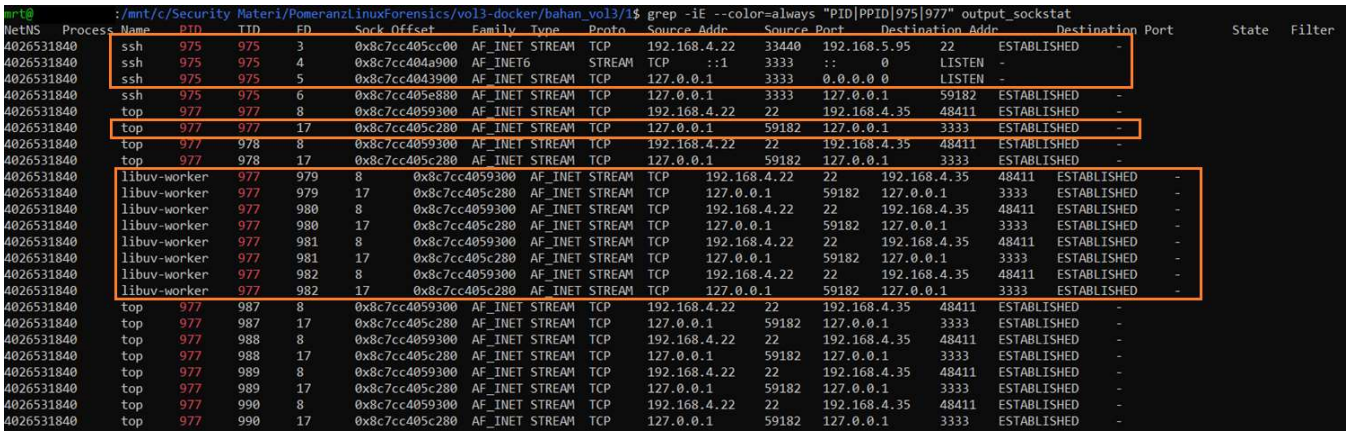


Figure F3-3. Process environment evidence showing that the payload was executed as ./top from /dev/shm/kit, with the SSH helper and payload sharing the same staging context.

Finding 4 — SSH helper and likely local port-forward workflow confirmed

Memory-resident socket evidence shows that **PID 975 (ssh)** maintained an outbound connection to 192.168.5.95:22 while also listening on 127.0.0.1:3333 and ::1:3333. At the same time, **PID 977 (top)** and related libuv-worker threads communicated with the same local endpoint. This strongly supports an **SSH-assisted local tunnel model**, in which the payload used 127.0.0.1:3333 as an intermediary communication path rather than connecting outward directly.

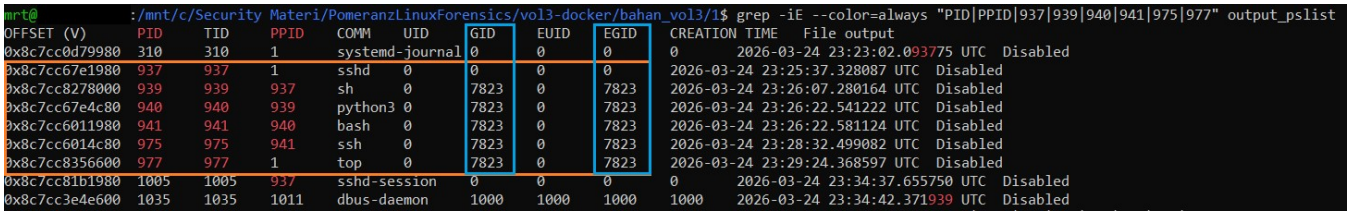


```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|975|977" output_sockstat
NetNS   Process Name    PID     TID     FD      Sock Offset     Family  Type    Proto   Source Addr     Source Port     Destination Addr        Destination Port        State   Filter
4026531840      ssh     975     975     3       0x8c7cc405cc00  AF_INET STREAM  TCP     192.168.4.22    33440   192.168.5.95    22      ESTABLISHED     -
4026531840      ssh     975     975     4       0x8c7cc404a900  AF_INET6        STREAM  TCP     ::1     3333    ::      0       LISTEN  -
4026531840      ssh     975     975     5       0x8c7cc4043900  AF_INET STREAM  TCP     127.0.0.1       3333    0.0.0.0 0       LISTEN  -
4026531840      ssh     975     975     6       0x8c7cc405e880  AF_INET STREAM  TCP     127.0.0.1       3333    127.0.0.1       59182   ESTABLISHED     -
4026531840      top     977     977     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     977     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     978     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     978     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     979     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     979     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     980     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     980     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     981     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     981     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      libuv-worker    977     982     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      libuv-worker    977     982     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     987     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     987     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     988     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     988     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     989     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     989     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
4026531840      top     977     990     8       0x8c7cc4059300  AF_INET STREAM  TCP     192.168.4.22    22      192.168.4.35    48411   ESTABLISHED     -
4026531840      top     977     990     17      0x8c7cc405c280  AF_INET STREAM  TCP     127.0.0.1       59182   127.0.0.1       3333    ESTABLISHED     -
```

Figure F4-1. Memory-resident socket evidence showing the SSH helper (PID 975) maintaining an outbound connection to 192.168.5.95:22, listening on 127.0.0.1:3333, and the top payload (PID 977) together with related libuv-worker threads communicating with the local tunnel endpoint.

Finding 5 — Attacker Process Chain Ran with Root Privileges and a Non-Standard Group Context

Memory-resident process listings show that the attacker-side chain — sh (939), python3 (940), bash (941), ssh (975), and top (977) — all ran as **UID 0**, confirming that the malicious workflow operated with **root privileges**. The same processes also shared **GID 7823** rather than the more typical **GID 0**. This is a noteworthy clustering characteristic, although the current evidence does not establish why that group context was used.



```
mrt@          :/mnt/c/Security Materi/PomeranzLinuxForensics/vol3-docker/bahan_vol3/1$ grep -iE --color=always "PID|PPID|937|939|940|941|975|977" output_pslist
OFFSET (V)      PID     TID     PPID    COMM    UID     GID     EUID    EGID    CREATION TIME   File output
0x8c7cc0d79980  310     310     1       systemd-journal 0       0       0       0       2026-03-24 23:23:02.093775 UTC  Disabled
0x8c7cc67e1980  937     937     1       sshd    0       0       0       0       2026-03-24 23:25:37.328087 UTC  Disabled
0x8c7cc8278000  939     939     937     sh      0       7823    0       7823    2026-03-24 23:26:07.280164 UTC  Disabled
0x8c7cc67e4c80  940     940     939     python3 0       7823    0       7823    2026-03-24 23:26:22.541222 UTC  Disabled
0x8c7cc6011980  941     941     940     bash    0       7823    0       7823    2026-03-24 23:26:22.581124 UTC  Disabled
0x8c7cc6014c80  975     975     941     ssh     0       7823    0       7823    2026-03-24 23:28:32.499082 UTC  Disabled
0x8c7cc8356600  977     977     1       top     0       7823    0       7823    2026-03-24 23:29:24.368597 UTC  Disabled
0x8c7cc81b1980  1005    1005    937     sshd-session    0       0       0       0       2026-03-24 23:34:37.655750 UTC  Disabled
0x8c7cc3e4e600  1035    1035    1011    dbus-daemon     1000    1000    1000    1000    2026-03-24 23:34:42.371939 UTC  Disabled
```

Figure F5-1. Process listing evidence showing that the attacker-side process chain ran as UID 0, while also sharing the non-standard group context GID 7823.

Finding 6 — Crash / reboot events still correlate with the suspicious activity window

The host experienced repeated crash/reboot events at:

- **12:56:39**
- **18:15:00**
- **19:16:24**

Of these, the **18:15** event is the most significant because it directly overlaps the longest and most relevant pre-acquisition worker session window, **14:54:59 - 18:15:05**. While the exact technical cause of the crashes remains unresolved, the timing strengthens the assessment that the host was materially affected during the malicious workflow.

Finding 7 — Userland anti-forensic behavior confirmed; active rootkit remains unproven

The reviewed evidence strongly supports **userland anti-forensic behavior**, including execution from **tmpfs** (/dev/shm/kit), **PATH manipulation**, deletion of the staging directory after launch, and detached execution of the final payload under **PID 1**. Together, these behaviors form a coherent workflow designed to reduce filesystem residue and complicate live-response triage. At the same time, the currently reviewed rootkit-check plugins do **not** establish the presence of an active kernel-level rootkit. The artifact **rk.so** remains suspicious and worth documenting, but it should presently be treated as a **suspicious but unconfirmed component**, rather than as a proven active part of the malicious execution chain.

Answers to SOC Questions

Q1. Why is there unencrypted traffic on 22/tcp?

The available host-side and memory-backed evidence confirms that **port 22/tcp was central to the observed workflow**, but it does **not conclusively establish why the NMS reportedly observed plaintext content on tcp/22**.

What can now be stated with confidence is that:

- the host was accessed through the known jump host **192.168.4.35**,
- that access was handled by **sshd**,
- an additional SSH-related helper process (**PID 975**) was launched from the suspicious staging directory,
- and that helper process connected to **192.168.5.95:22** while exposing a local endpoint that was then used by the payload.

The most defensible interpretation is therefore that **port 22 was operationally important because it carried both SSH-based access and SSH-assisted tunnel activity**, not because a fake service was bound to port 22.

At the same time, memory analysis now confirms that the exact command referenced in the alert was executed on the host by **PID 940**: **`python3 -c import pty; pty.spawn("/bin/bash")`** .This means the host artifacts now directly corroborate the alert content itself. In addition, port 22 was not only used by sshd, but was central to both the attacker-side interactive shell chain and the later acquisition-related session.

That said, the host-side artifacts still do **not** identify the exact point in the network path where the command content was observed in plaintext on tcp/22. What they do establish is that the command was executed in an **SSH-derived session on the victim host**, and that port 22 was integral to the attacker's workflow.

Q2. Why is the system burning CPU?

The CPU burn is best explained by **PID 977**, the disguised **top** payload executed from **/dev/shm/kit**. Multiple sources of evidence support this conclusion, including **Bash history, PsAux, Maps, Envars, and UAC artifacts**.

The strongest indicators are:

- the explicit command history showing **mv xmrig top**
- execution of the payload as **top -o 127.0.0.1:3333 -B**
- the fact that the process later ran under **PPID 1**
- and process evidence showing **99% CPU utilization**

In addition, the payload was backed in memory by **/dev/shm/kit/top** and spawned multiple **libuv-worker** threads, which is consistent with a high-activity userland workload such as **cryptomining**. Taken together, this evidence is now strong enough to treat the miner hypothesis as the primary explanation for the reported CPU exhaustion.

Q3. Why can't the SOC see what is going on?

The visibility gap is best explained by a combination of several confirmed behaviors:

- execution from **tmpfs** (/dev/shm/kit)
- renaming the payload from **xmrig** to **top**

- modifying **PATH** so the local binary would be preferred
- detached or orphaned runtime under **PID 1**
- and deletion of the staging directory after launch

By the time UAC captured the system, the payload was still visible in process-related artifacts, but its underlying working directory and executable path had already become **deleted tmpfs paths**. That significantly reduced the visibility available through normal live-response review and made routine process triage much more difficult.

The earlier description of this as a **"hidden process symptom"** can still be retained, but it is now better understood as the result of a confirmed **masquerading, tmpfs staging, detached execution, and cleanup workflow**, rather than as an unexplained anomaly on its own.

Q4. Is the system compromised?

Based on the UAC artifacts and memory-backed evidence reviewed so far, the system is **very likely compromised**.

This assessment is supported by multiple converging indicators, including:

- a suspicious process cluster associated with **/dev/shm** and **/dev/shm/kit**
- explicit in-memory command history showing **payload preparation and launch**
- **PID 975 (ssh)** operating from the same deleted staging directory
- **PID 977** resolving to **/dev/shm/kit/top (deleted)**
- execution from **deleted tmpfs-backed paths**
- confirmed tunnel-like communication behavior involving **127.0.0.1:3333** and **192.168.5.95:22**
- and repeated **crash/reboot events** overlapping the main suspicious activity window

Taken together, these artifacts do not reflect a benign or routine administrative scenario. Instead, they support a high-confidence conclusion that the host was actively used in a malicious workflow involving staged execution, masquerading, SSH-assisted communication, and operational impact on the system.

Q5. When and how did that happen?

The most relevant pre-acquisition activity window remains **2026-03-24 14:54:59–18:15:05**, corresponding to the longest recorded remote **worker** session on **pts/1**, followed by a concurrent **worker** session on **pts/3** shortly before the second crash/reboot at **18:15:00**. This remains the strongest candidate window for the primary malicious activity.

Based on the evidence reviewed so far, the most defensible reconstruction is as follows:

1. access to the host occurred through the normal SSH path via the jump host;
2. the attacker established an interactive shell chain;
3. tooling was staged in **/dev/shm/kit**;
4. **xmrig** was renamed to **top**;
5. **PATH** was modified so the local binary would be preferred;
6. the SSH helper was launched with **ssh -F config ymv**;
7. the disguised payload was executed as **top -o 127.0.0.1:3333 -B**;
8. the staging directory was deleted;
9. the payload continued running as an orphaned process under **PID 1**; and

10. the host then experienced repeated crash/reboot events, including one that directly overlapped the main suspicious session window.

In short, the compromise appears to have unfolded as an SSH-derived interactive operation that progressed into staged payload execution from tmpfs, followed by masquerading, SSH-assisted tunnel activity, anti-forensic cleanup, and continued detached execution of the final payload.

IOC / Indicators

A. Host-Based Indicators

- **`/dev/shm/kit/`**
  Suspicious staging directory used by the threat actor to store the toolkit and payload.
- **`/dev/shm/kit/top`**
  Path of the primary payload executed from tmpfs and associated with **PID 977**. In UAC artifacts, this path appears as **`/dev/shm/kit/top (deleted)`**.
- **`/dev/shm/kit (deleted)`**
  Deleted working directory still associated with:
  - **PID 975** (`/usr/bin/ssh`)
  - **PID 977** (`top`)
- **`/dev/shm`**
  Volatile staging location relevant to this case, particularly because:
  - **PID 941** (`bash`) had `cwd -> /dev/shm`
  - the threat actor moved into **`/dev/shm/kit`**
  - the payload was executed from this area
- **Primary process indicators**
  - **PID 939** → `sh`
  - **PID 940** → `python3.13`
  - **PID 941** → `bash`
  - **PID 975** → `/usr/bin/ssh`
  - **PID 977** → `top` (disguised payload executed from `/dev/shm/kit/top`)
- **Attacker process chain**
  - `sshd (937) -> sh (939) -> python3.13 (940) -> bash (941) -> ssh (975)`
  - followed by **`top (977)`** continuing as an orphaned process under **PID 1**
- **Command indicators confirmed in memory**
  - `cd /dev/shm/kit`
  - `mv xmrig top`
  - `export PATH=.:$PATH`
  - `ssh -F config ymv`
  - `top -o 127.0.0.1:3333 -B`
  - `rm -rf kit`
- **Masquerading indicator**
  - `xmrig` was renamed to `top`
- **Execution context indicators**
  - the payload was executed as **`./top`**
  - **`PWD=/dev/shm/kit`**
  - **`PATH=.:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin`**

B. Network Indicators

- **127.0.0.1:3333**
  Local endpoint used in the payload's operational workflow. Memory evidence shows that:
  - **PID 975** (ssh) opened a listener on **127.0.0.1:3333**
  - **PID 977** (top) and related threads communicated with this endpoint
- **::1:3333**
  Additional local IPv6 listener also opened by **PID 975** (ssh)
- **192.168.5.95:22**
  Suspicious remote SSH endpoint involved in the observed workflow. At this stage, it is most appropriately treated as:

    - a **suspicious remote SSH endpoint**
    - a **high-value investigative indicator**
    - **not yet** something that should be labeled definitively as the final C2 without additional validation
- **Relevant account / operational strings**
    - pocmp@192.168.5.95
    - ssh -F config ymv
- **Most plausible communication model**
    - top -> 127.0.0.1:3333 -> ssh helper -> 192.168.5.95:22

C. Contextual / Supporting Indicators

- **Repeated crash / reboot events**
    - 2026-03-24 12:56:39
    - 2026-03-24 18:15:00
    - 2026-03-24 19:16:24
- **Package installation activity by worker**
    - build-essential
    - git
    - libpam0g-dev
    - libgcrypt-dev
    - nasm

  These indicators are important for investigative context, but they are **not standalone IOCs**.
- **Process visibility anomaly**
    - **PID 977** remained referenced in process-related artifacts
    - the executable path and working directory had already become **(deleted)**
    - the payload continued running as an orphaned process under **PID 1**
    - together, these conditions support the **visibility gap** reported by the SOC
- **Additional staging-file artifacts worth noting**
    - config
    - rk.so
    - xmrig

  With respect to **rk.so**, it should currently be treated as a **suspicious artifact found in the staging directory**, but it has **not yet been confirmed** as an actively loaded component in the payload chain observed so far.

D. Not IOC / Do Not Mislabel

- **192.168.4.35**
  This is the **jump host / infrastructure hop**, not a malicious IP.
- **Port 22/tcp**
  Relevant as the **operational SSH channel** in this case, but port 22 itself is **not a malicious IOC**.
- **sshd**
  The SSH service itself remains a normal system service. What is suspicious in this case is **how the SSH path was used within the attacker's workflow**, not the mere presence of sshd.

Conclusion

The available evidence strongly supports the conclusion that the Linux worker host was used as a **cryptomining node**, with a disguised payload staged and executed from **/dev/shm/kit**. Taken together, the UAC artifacts and confirmed memory findings now provide a much clearer and more complete narrative than was possible in the earlier draft. The threat actor worked from a **root shell**, staged tooling in **tmpfs**, renamed **xmrig** to **top**, modified **PATH** so the local binary would be invoked, launched an **SSH helper** with a custom configuration, used the local endpoint **127.0.0.1:3333** as part of the runtime path, and removed the staging directory after launch. The payload then continued running as **PID 977** under **PID 1**, consistent with the **CPU-burn** and **visibility-gap** symptoms reported by the SOC.

At the same time, the conclusion should remain disciplined in two important respects. First, **192.168.4.35** remains a **jump host**, not a malicious IOC. Second, **192.168.5.95** is now clearly relevant as a **suspicious remote SSH endpoint involved in the observed workflow**, but the report does not need to overstate it as a definitively proven final **C2 server**. Likewise, **rk.so** remains noteworthy as a suspicious artifact found in the staging directory, but it should continue to be framed as an **unconfirmed actively loaded malicious component** unless further evidence establishes that role more directly.